# Building Human-Centered AI Applications

## Lab: Re-ranking a RAG Pipeline

*Group 6 topic presentation: RAG and Re-ranking (November 10)*

**Goal:** Our RAG app already finds abstracts for a claim. You will add a re-ranker, a second step that re-sorts what the search found. Then you will measure whether it finds better abstracts, and whether that changes the LLM's answer.

**Data:** SciFact (Wadden et al., 2020): 5,183 abstracts from medical research papers and 300 test claims. For each claim, experts listed the abstracts that support or refute it. We call that list the answer key. The data is in the starter's data/scifact folder.

### Words used in this lab

**Chunk:** a short piece of an abstract, about 120 words. A long abstract is split into several chunks.

**Stage 1 search (already built):** turns the claim and every chunk into lists of numbers (embeddings) and returns the chunks whose numbers are closest to the claim's. It is fast but rough.

**Re-ranker (stage 2, you build it):** a cross-encoder model that reads the claim and one chunk together and gives a more careful relevance score. It only re-sorts the chunks stage 1 found. It never searches by itself.

**Recall@k:** of the abstracts in the answer key, the share that shows up in the top k results. 1.0 means all of them made it.

**MRR (mean reciprocal rank):** for each claim, 1 divided by the position of the first correct abstract (1st place scores 1, 2nd scores 0.5, 3rd scores 0.33), averaged over all claims. Higher is better.

**Latency:** how long a step takes, shown here in milliseconds.

### Setup (5 minutes)

1. Copy LabRerank.py and the data folder from the starter into your app's repository. Add LabRerank.py to your app's navigation as a new page.
2. Copy the lines from the starter's requirements.txt into yours. Keep the --extra-index-url line: it makes pip install the small CPU version of PyTorch. Then push and deploy right away. The first Streamlit Cloud build takes several minutes because it installs PyTorch, so let it run while you work on your laptop.
3. Run the page on your laptop. The first run downloads two small models and builds the search index from about 12,477 chunks, which takes about 90 seconds. After that the results are cached and the page loads quickly.

### Part A: Look at the stage 1 search (5 minutes)

4. Read retrieve() and build_index(). Then pick a few test claims from the dropdown. The app shows the top 3 abstracts and puts a green tick next to each one that is in the answer key. Write down any claim that has no green tick in its top 3.
5. In a comment at the top of your file, answer two questions: why does build_index() put the title in front of every chunk, and why is to_doc_ids() needed?

**Part B: Add the re-ranker (10 minutes)**

6. Write the code for rerank(). The re-ranker takes (query, passage) pairs and returns one score per pair. This is how it is used:

```
from sentence_transformers import CrossEncoder

model = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
scores = model.predict([("my query", "first passage"),
                        ("my query", "second passage")])
# one score per pair; higher means more relevant
```

   a. Build one (claim, chunk text) pair for every hit and score them all in one predict() call. Do not call predict() in a loop.
   b. Return the hits sorted by the new score, highest first, in the same (chunk_index, score) format they came in.

7. Go back to the claims you wrote down in step 4. Find one where re-ranking brings a green tick into the top 3 (for example: "Insulin increases risk of severe kidney failure."). Then find one where re-ranking changes nothing or makes the result worse.

**Part C: Measure before and after (10 minutes)**

8. Write the code for evaluate(). Use the given recall_at_k() and reciprocal_rank() to score the results with and without re-ranking. Time stage 1 and stage 2 separately with time.perf_counter().
9. Add a button that runs evaluate() and shows three numbers with st.metric: Recall@k, MRR, and the extra time re-ranking adds. Use delta= so each number also shows how much re-ranking changed it.
10. Set the Stage 1 candidates slider to 5, 10, 30 and 100, and run the evaluation each time. In your comment, explain why re-ranking helps so little at 5 candidates, and what it costs you at 100.

**Part D: Write the prompt (10 minutes)**

11. Write SYSTEM_TEMPLATE, the system prompt. It must tell the model to use only the abstracts it is given, to give exactly one verdict (SUPPORTED, REFUTED, or NOT ENOUGH EVIDENCE), and to cite the abstracts it used by their [id].
12. Write the code for verify_claim(): one OpenAI chat call, as in Lab 4. Then add a button that shows two answers side by side: one from the abstracts found without re-ranking, and one from the abstracts found with it.
13. Find a claim where the two answers differ. Which one is right, and what made the difference? Remember: a re-ranker scores how relevant an abstract is, not whether it agrees with the claim. An abstract that refutes the claim is still relevant evidence.

**Optional**

Change RERANK_MODEL to "BAAI/bge-reranker-base", a model about twelve times larger, and run Part C again. Does the bigger model do better on these claims? Is it worth the extra time?

**Tip:** If the Streamlit Cloud build runs out of memory or takes very long, check that requirements.txt has the --extra-index-url line. Without it, pip installs the much larger GPU version of PyTorch.

To submit: upload LabRerank.py to Blackboard (with your comment answering steps 5, 10 and 13) and include the link to your app.